**FUNDAÇÃO UNIVERSIDADE FEDERAL DE RONDÔNIA - UNIR**

Faculdade de Engenharia, Tecnologia e Inovação - FAETI
Bacharelado em Ciência da Computação

**PROCESSAMENTO DE IMAGENS**

# Relatório da Primeira Avaliação

Biblioteca de Processamento de Imagens em GNU Octave

**Ricardo Alves da Silva**

Matrícula: 20231008324

Professor: Lucas Marques da Cunha

Porto Velho - RO
2026

## Identificação do trabalho

| | |
|---|---|
| **Aluno(a):** | Ricardo Alves da Silva |
| **Matrícula:** | 20231008324 |
| **Disciplina:** | Processamento de Imagens |
| **Professor:** | Lucas Marques da Cunha |
| **Título do sistema:** | Biblioteca de processamento de imagens em escala de cinza (8 bits) |
| **Linguagem/ambiente:** | GNU Octave 11.3 (linha de comando), ambiente reprodutível com Nix |
| **Repositório:** | https://github.com/riccardoalv/pdi-av1 |

## Sumário


**1 Introdução** **3**

1.1 Objetivos . . . 3

**2 Fundamentação teórica** **3**

2.1 Representação de imagens digitais e vizinhança . . . 3

2.2 Operações algébricas e transformações de intensidade . . . 4

2.3 Histograma . . . 4

2.4 Filtragem espacial e convolução . . . 5

2.5 Detecção de bordas . . . 5

2.6 Aguçamento . . . 5

2.7 Controle de contraste adaptativo . . . 6

**3 Materiais e métodos** **6**

3.1 Ambiente de desenvolvimento . . . 6

3.2 Arquitetura e organização do sistema . . . 6

3.3 Implementação das operações . . . 7

3.3.1 Operações algébricas e transformações de intensidade . . . 7

3.3.2 Expansão e equalização de histograma . . . 7

3.3.3 Filtros da média e mediana . . . 8

3.3.4 Convolução genérica . . . 8

3.3.5 Roberts, Sobel, Kirsch e Laplaciano . . . 8

3.3.6 Aguçamento e high boost . . . 8

3.3.7 Controle de contraste adaptativo . . . 9

**4 Resultados** **9**

4.1 Utilização da biblioteca . . . 9

4.2 Transformações de intensidade e histogramas . . . . . 9
4.3 Filtragem espacial . . . . . 12
4.4 Testes de convolução . . . . . 13
4.4.1 Filtro de aguçamento . . . . . 13
4.4.2 Filtros de relevo . . . . . 14
4.4.3 Máscaras de detecção de bordas . . . . . 14
4.5 Roberts, Sobel, Kirsch e Laplaciano . . . . . 15
4.6 Aguçamento pelo Laplaciano e high boost . . . . . 16
4.7 Controle de contraste adaptativo . . . . . 17

**5 Discussão** **17**
5.1 Questões de domínio conceitual . . . . . 18
5.2 Limitações e dificuldades encontradas . . . . . 19

**6 Conclusão** **20**

**7 Declaração de uso de ferramentas de Inteligência Artificial** **20**

**8 Referências** **20**

# 1 Introdução

O processamento digital de imagens trata uma imagem como uma função discreta $f(x, y)$, em que cada par de coordenadas corresponde a um pixel e o valor da função corresponde à intensidade naquele ponto. Em imagens em escala de cinza de 8 bits, essa intensidade é um inteiro no intervalo $[0, 255]$, sendo 0 o preto e 255 o branco. Manipular esses valores permite corrigir problemas de aquisição, como baixo contraste, ruído e falta de nitidez, e também extrair informações estruturais, como as bordas dos objetos.

As operações estudadas na Unidade 01 dividem-se em dois grandes grupos. As *transformações pontuais* alteram cada pixel considerando apenas o seu próprio valor: são o negativo, a limiarização, as transformações de potência e logarítmica, o alargamento de contraste e as operações baseadas em histograma. As *operações de vizinhança* consideram também os pixels ao redor: são os filtros espaciais, a convolução, os detectores de bordas e as técnicas de aguçamento. A convolução tem papel central, pois um mesmo procedimento, aplicado com máscaras diferentes, produz suavização, realce de bordas, efeito de relevo ou aguçamento.

Este trabalho apresenta uma biblioteca em GNU Octave que implementa essas operações sem recorrer a funções prontas de processamento: as bibliotecas da linguagem foram usadas apenas para leitura e gravação das imagens. Cada operação é uma função independente, coberta por testes unitários com valores calculados manualmente, o que permite verificar o comportamento de cada algoritmo em pequenas matrizes de pixels.

## 1.1 Objetivos

**Objetivo geral.** Implementar e analisar, em GNU Octave, as operações de processamento de imagens em escala de cinza de 8 bits da Unidade 01, sem utilizar funções prontas para os algoritmos avaliados.

**Objetivos específicos.**

- Implementar uma convolução genérica $g = f * h +$offset com máscara $n \times m$, tratamento explícito das bordas e dos valores fora de $[0, 255]$.
- Implementar as operações algébricas (dissolve cruzado uniforme e não uniforme) e as transformações de intensidade (negativo, alargamento de contraste, limiarização, potência e logaritmo).
- Implementar o cálculo, a expansão e a equalização de histograma, o contraste adaptativo e os filtros da média e da mediana.
- Implementar os detectores de bordas de Roberts, Sobel, Kirsch e Laplaciano, exibindo as respostas intermediárias, além do aguçamento pelo Laplaciano e da técnica *high boost*.
- Validar todas as funções com testes unitários e analisar os resultados em imagens reais, incluindo os testes obrigatórios de convolução.

# 2 Fundamentação teórica

## 2.1 Representação de imagens digitais e vizinhança

Uma imagem digital em escala de cinza é uma matriz $M \times N$ em que cada elemento $f(x, y)$ representa a intensidade de um pixel. Com 8 bits por pixel, há $L = 2^8 = 256$ níveis possíveis, no intervalo $[0, L - 1] = [0, 255]$.

A *vizinhança* de um pixel é o conjunto de pixels ao seu redor. A vizinhança-4 contém os quatro vizinhos horizontais e verticais; a vizinhança-8 acrescenta os quatro diagonais. Operações de

vizinhança usam uma janela $n \times m$ centrada no pixel. Nas bordas da imagem, parte da janela fica fora dos limites, e é preciso decidir como preencher esses valores: com zeros, com uma constante, repetindo o pixel da borda, entre outras estratégias. Essa escolha afeta apenas uma faixa de largura $\lfloor n/2 \rfloor$ junto às margens, mas pode introduzir bordas artificiais quando o preenchimento difere muito do conteúdo da imagem.

## 2.2 Operações algébricas e transformações de intensidade

O *dissolve cruzado* combina duas imagens do mesmo tamanho por média ponderada. No caso uniforme, o peso $t \in [0, 1]$ é o mesmo para todos os pixels:

$$g(x, y) = (1 - t)\, f_1(x, y) + t\, f_2(x, y). \tag{1}$$

No caso não uniforme, cada pixel tem seu próprio peso, dado por uma matriz $T(x, y) \in [0, 1]$:

$$g(x, y) = \big(1 - T(x, y)\big)\, f_1(x, y) + T(x, y)\, f_2(x, y). \tag{2}$$

Como se trata de uma combinação convexa, o resultado nunca sai do intervalo $[0, 255]$.

As transformações de intensidade são funções $s = T(r)$ aplicadas a cada pixel de valor $r$:

- **Negativo:** $s = (L-1) - r = 255 - r$. Inverte a escala de cinza e facilita a observação de detalhes claros em fundos escuros.
- **Limiarização:** $s = 255$ se $r \geq t$ e $s = 0$ caso contrário. Produz uma imagem binária que separa objeto e fundo.
- **Potência (gamma):** $s = c\, r^{\gamma}$, com $r$ normalizado em $[0, 1]$. Para $\gamma < 1$ os tons escuros são expandidos e a imagem clareia; para $\gamma > 1$ os tons claros são expandidos e a imagem escurece; $\gamma = 1$ com $c = 1$ é a identidade.
- **Logarítmica:** $s = c\, \log(1 + r)$. Expande fortemente os tons escuros e comprime os claros; com $c = 255/\log 256$, o intervalo $[0, 255]$ é mapeado em si mesmo.
- **Alargamento de contraste:** função linear por partes que passa pelos pontos $(0, 0)$, $(r_1, s_1)$, $(r_2, s_2)$ e $(255, 255)$. Nos trechos com inclinação maior que 1 o contraste aumenta; nos trechos com inclinação menor que 1, diminui.

## 2.3 Histograma

O histograma $h(r_k) = n_k$ conta quantos pixels possuem cada nível $r_k$. Dividindo pelo total de pixels $MN$, obtém-se a probabilidade $p(r_k) = n_k/MN$.

A *expansão* de histograma é uma transformação linear que leva o menor nível presente na imagem a 0 e o maior a 255:

$$s = \frac{r - r_{\min}}{r_{\max} - r_{\min}} \cdot 255. \tag{3}$$

Ela estica o histograma sem alterar sua forma.

A *equalização* usa a distribuição acumulada para redistribuir os níveis:

$$s_k = (L - 1) \sum_{j=0}^{k} p(r_j). \tag{4}$$

Cada nível é levado à fração de pixels com intensidade menor ou igual a ele, de modo que os níveis com muitos pixels ficam mais afastados entre si. O histograma resultante tende a ser aproximadamente uniforme.

## 2.4 Filtragem espacial e convolução

A convolução de uma imagem $f$ com uma máscara $h$ de dimensão $n \times m$ é definida por

$$g(x,y) = \sum_s \sum_t h(s,t)\, f(x-s,\, y-t) + \text{offset}. \tag{5}$$

O sinal negativo nos índices equivale a rotacionar a máscara em $180°$ antes de aplicá-la. Sem essa rotação, a operação é chamada correlação. Para máscaras simétricas em $180°$ as duas operações coincidem; para máscaras direcionais, como as de relevo, elas produzem resultados espelhados.

A convolução é *linear*: $h * (af_1 + bf_2) = a(h * f_1) + b(h * f_2)$. O *filtro da média* é uma convolução com a máscara $n \times n$ de coeficientes $1/n^2$, cuja soma é 1, o que preserva o brilho médio. O *filtro da mediana* substitui o pixel pelo valor central dos valores ordenados da janela. A mediana não é linear, portanto não pode ser expressa como convolução.

A soma dos coeficientes da máscara indica o comportamento esperado em regiões homogêneas: soma 1 preserva o brilho; soma 0 anula regiões planas e destaca apenas variações; soma maior que 1 clareia; soma negativa inverte o brilho.

## 2.5 Detecção de bordas

Bordas são regiões de variação abrupta de intensidade e podem ser detectadas por derivadas. Os operadores de primeira ordem estimam o gradiente $\nabla f = (G_x, G_y)$, e a intensidade da borda é a magnitude

$$|\nabla f| = \sqrt{G_x^2 + G_y^2}. \tag{6}$$

- **Roberts:** duas máscaras $2 \times 2$ que calculam diferenças diagonais, $\left[\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right]$ e $\left[\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right]$. São simples e rápidas, mas muito sensíveis a ruído.
- **Sobel:** duas máscaras $3 \times 3$, $G_x = \left[\begin{smallmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{smallmatrix}\right]$ e $G_y = \left[\begin{smallmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{smallmatrix}\right]$. O peso 2 na linha ou coluna central combina a derivada com uma suavização na direção perpendicular, o que reduz a sensibilidade ao ruído.
- **Kirsch:** oito máscaras $3 \times 3$, uma para cada direção (N, NE, E, SE, S, SO, O, NO), obtidas por rotações de $45°$ de $\left[\begin{smallmatrix} 5 & 5 & 5 \\ -3 & 0 & -3 \\ -3 & -3 & -3 \end{smallmatrix}\right]$. A resposta final é o máximo entre as oito respostas.
- **Laplaciano:** derivada de segunda ordem, $\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$, aproximada pelas máscaras $\left[\begin{smallmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{smallmatrix}\right]$ (vizinhança-4) e $\left[\begin{smallmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{smallmatrix}\right]$ (vizinhança-8). É isotrópico (não tem direção), responde com sinais opostos de cada lado de uma borda e é bastante sensível a ruído.

## 2.6 Aguçamento

O aguçamento realça transições de intensidade somando à imagem original uma componente que só existe nas bordas. No aguçamento pelo Laplaciano,

$$g = f + c\,\nabla^2 f, \tag{7}$$

com $c = -1$ quando o centro da máscara é negativo e $c = +1$ quando é positivo. Em ambos os casos o efeito é aumentar a diferença entre o pixel e seus vizinhos.

Na técnica *high boost*, a componente de detalhe é obtida subtraindo da imagem uma versão suavizada:

$$g = f + k\,\big(f - \bar{f}\big), \tag{8}$$

em que $\bar{f}$ é a imagem filtrada pela média. Com $k = 1$ obtém-se o *unsharp masking*; com $k > 1$, o realce é mais intenso.

### 2.7 Controle de contraste adaptativo

O contraste adaptativo ajusta cada pixel de acordo com as estatísticas da sua vizinhança $n \times n$: a média local $m(x, y)$ e o desvio padrão local $\sigma(x, y)$. A formulação adotada neste trabalho é

$$g(x, y) = m(x, y) + \frac{c}{\sigma(x, y)} \big(f(x, y) - m(x, y)\big), \tag{9}$$

em que $c$ é uma constante que define o contraste desejado. O termo $f - m$ mede o quanto o pixel se destaca da vizinhança, e o ganho $c/\sigma$ amplia esse destaque em regiões de baixa variação e o reduz em regiões de alta variação. Com isso, o contraste local tende a $c$ em toda a imagem.

## 3 Materiais e métodos

### 3.1 Ambiente de desenvolvimento

O sistema foi implementado em GNU Octave 11.3 em Linux. O ambiente é descrito em um arquivo `flake.nix`, e o comando `nix develop` disponibiliza exatamente as mesmas versões das ferramentas em qualquer máquina:

- GNU Octave 11.3 com o pacote `image`, usado apenas para leitura e gravação de imagens;
- MISS_HIT, para verificação de estilo (`mh_style`) e análise estática (`mh_lint`);
- GNU Make, para automatizar testes, verificação de estilo, geração dos resultados e compilação deste relatório.

Das bibliotecas do Octave foram usadas apenas `imread`, `imwrite` e `rgb2gray` (leitura, gravação e conversão de imagens coloridas), além de operações matemáticas elementares (`sum`, `sort`, `sqrt`, `log`, `cumsum`, `max`). Nenhum algoritmo avaliado foi implementado com funções prontas como `conv2`, `imfilter`, `histeq`, `imadjust`, `medfilt2` ou `edge`.

### 3.2 Arquitetura e organização do sistema

O sistema é uma biblioteca: cada operação é uma função em um arquivo próprio na pasta `src/`. As funções são independentes e reaproveitam umas às outras. A Figura 1 mostra as dependências.

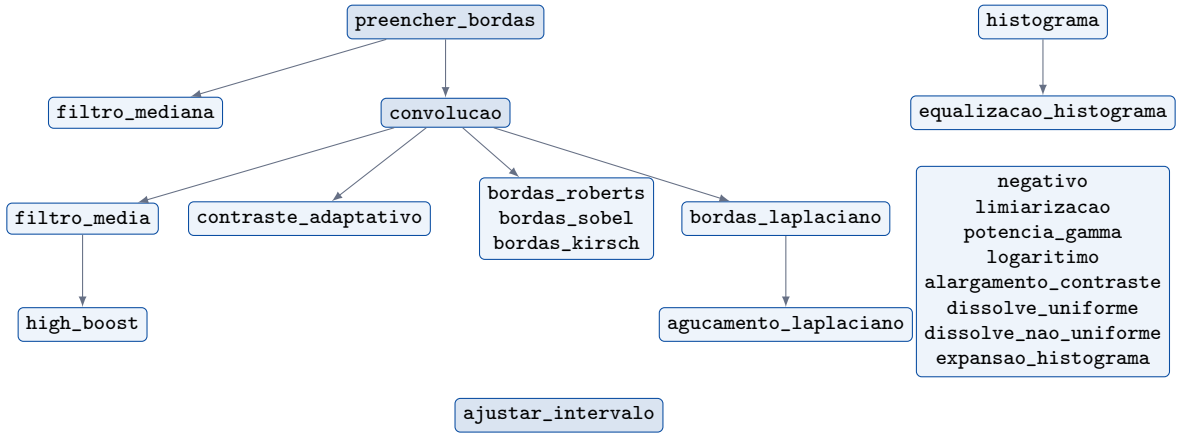



**Figura 1:** Dependências entre as funções da biblioteca. A seta indica que a função de destino usa a de origem. As funções da coluna da direita são pontuais e não dependem da convolução.

O repositório está organizado da seguinte forma:

- `src/`: as 22 funções da biblioteca;
- `tests/`: um arquivo de testes por função e o executor `run_tests.m`;
- `gerar_resultados.m`: aplica as operações às imagens de `images/` e grava os resultados em `resultados/`;
- `images/`: imagens de entrada; `resultados/`: imagens produzidas; `docs/`: enunciado e relatório.

Os testes usam os blocos nativos do Octave (`%!test`, `%!assert`, `%!error`) e comparam a saída de cada função com valores calculados à mão em matrizes pequenas. São 184 testes, executados com `make test`; `make check` executa também a verificação de estilo e a análise estática.

## 3.3 Implementação das operações

Três decisões de projeto valem para toda a biblioteca:

1. **Cálculo em ponto flutuante.** As contas são feitas em `double`, e a conversão para `uint8` ocorre apenas no final. Em `uint8`, cada operação intermediária satura e arredonda: `uint8(200)+uint8(100)` resulta em 255, e `uint8(64)/255` resulta em 0.
2. **Tratamento explícito de $[0, 255]$.** A função `ajustar_intervalo(r, modo)` converte o resultado real para `uint8` em três modos: `saturar` (valores negativos viram 0 e acima de 255 viram 255; é o padrão), `normalizar` (reescala linear de $[\min, \max]$ para $[0, 255]$) e `modulo` (valor absoluto seguido de saturação). Em todos os modos o resultado é arredondado com `round`.
3. **Tratamento explícito das bordas.** A função `preencher_bordas(f, tamanho, modo, constante)` acrescenta as linhas e colunas necessárias para a máscara nos modos `zero`, `constante` e `repetir`. Qualquer modo não reconhecido é tratado como `zero`.

### 3.3.1 Operações algébricas e transformações de intensidade

Todas são pontuais e foram implementadas de forma vetorizada, operando sobre a matriz inteira:

- `negativo`: `255 - imagem`. O resultado nunca sai do intervalo.
- `limiarizacao(imagem, limiar)`: pixels menores que o limiar recebem 0, e os demais recebem 255.
- `potencia_gamma(imagem, c, y)`: normaliza $r$ para $[0, 1]$, calcula $255\, c\, r^{\gamma}$ com potência elemento a elemento (`.^`) e ajusta o intervalo. A função não se chama `gamma` porque esse nome pertence à função $\Gamma$ do Octave, usada internamente por `factorial`.
- `logaritimo(imagem, c)`: $c \log(1 + r)$, seguida do ajuste do intervalo.
- `alargamento_contraste(imagem, r1, s1, r2, s2)`: três máscaras lógicas selecionam os pixels de cada trecho ($r < r_1$, $r_1 \leq r < r_2$ e $r \geq r_2$), e cada grupo recebe a equação da reta correspondente. O caso $r_2 = 255$ é tratado à parte para evitar divisão por zero.
- `dissolve_uniforme` e `dissolve_nao_uniforme`: convertem as imagens para `double` antes da soma ponderada; na versão não uniforme, os produtos são elemento a elemento (`.*`).

### 3.3.2 Expansão e equalização de histograma

A função `histograma` percorre os 256 níveis e conta quantos pixels são iguais a cada um; o nível $k$ é armazenado na posição $k+1$, pois os índices do Octave começam em 1. A `expansao_histograma` aplica a equação da expansão por meio de `ajustar_intervalo(imagem, 'normalizar')`; uma imagem constante, em que $r_{\max} = r_{\min}$, é devolvida sem alteração. A `equalizacao_histograma`

calcula as probabilidades, a distribuição acumulada (`cumsum`) e uma tabela de 256 posições com $255 \cdot cdf$; cada pixel é então substituído pelo valor da tabela correspondente ao seu nível.

### 3.3.3 Filtros da média e mediana

O `filtro_media(imagem, n, modo)` reaproveita a convolução com a máscara `ones(n)/(n*n)`. O `filtro_mediana(imagem, n, modo)` preenche as bordas, percorre cada pixel, recorta a janela $n \times n$, ordena seus valores com `sort` e seleciona o elemento central, de posição $(n^2 + 1)/2$. O tamanho da vizinhança e o modo de borda são parâmetros.

### 3.3.4 Convolução genérica

A função `convolucao(imagem, kernel, modo, offset, constante)` implementa $g = f * h +$ offset para máscaras $n \times m$ quaisquer, inclusive retangulares e de dimensão par:

1. a máscara é rotacionada em $180°$ com `kernel(end:-1:1, end:-1:1)`, o que caracteriza a convolução (e não a correlação);
2. a imagem é convertida para `double` e preenchida por `preencher_bordas`. Para cada dimensão de tamanho $n$, são acrescentadas $\lceil n/2 \rceil - 1$ posições antes e $\lfloor n/2 \rfloor$ depois; para $n$ ímpar isso equivale a $(n-1)/2$ de cada lado, e para $n$ par o centro da máscara é a posição $\lceil n/2 \rceil$;
3. para cada pixel $(i, j)$, a janela da imagem preenchida é `(i:i+n-1, j:j+m-1)`, que é multiplicada elemento a elemento pela máscara; a soma dos produtos, acrescida do offset, é o valor de saída.

O resultado é devolvido em `double` e pode conter valores negativos ou maiores que 255. A conversão para imagem é feita em seguida por `ajustar_intervalo`, no modo escolhido. Os parâmetros `modo`, `offset` e `constante` são opcionais, com padrões `zero`, 0 e 128; as funções que dependem da convolução repassam esses parâmetros com `varargin`.

### 3.3.5 Roberts, Sobel, Kirsch e Laplaciano

Cada detector aplica suas máscaras com a função `convolucao` e devolve uma estrutura com todas as respostas intermediárias:

- `bordas_roberts` e `bordas_sobel`: campos `gx`, `gy`, `magnitude` $= \sqrt{G_x^2 + G_y^2}$ e `final` (magnitude saturada em `uint8`);
- `bordas_kirsch`: as oito máscaras, a matriz `respostas` ($M \times N \times 8$), o `maximo` entre elas, a `direcao` (índice da máscara vencedora) e o `final`;
- `bordas_laplaciano(imagem, vizinhanca)`: a `mascara` de vizinhança 4 ou 8, a `resposta` com sinal e o `final`, convertido no modo `modulo` para que os dois lados de cada borda apareçam.

Como a convolução gira a máscara, $G_x$ do Sobel é negativo quando a intensidade cresce da esquerda para a direita, e no Kirsch a máscara de rótulo O (oeste) responde às bordas cujo lado claro está a leste. A magnitude e o máximo final não são afetados.

### 3.3.6 Aguçamento e high boost

`agucamento_laplaciano(imagem, vizinhanca)` usa `bordas_laplaciano` e devolve separadamente a resposta do Laplaciano (`laplaciano`, com sinal, e `laplaciano_visual`, normalizada para exibição) e a imagem combinada `final` $= f - \nabla^2 f$. A subtração decorre do centro negativo das máscaras.

`high_boost(imagem, k, n)` obtém a versão suavizada com `filtro_media`, calcula a máscara de detalhe $f - \bar{f}$ e devolve $f + k(f - \bar{f})$. Como o filtro da média já devolve a imagem arredondada, a máscara de detalhe tem erro de até $\pm 0{,}5$, que é multiplicado por $k$; para $k = 1$ esse erro não altera o resultado inteiro final.

### 3.3.7 Controle de contraste adaptativo

`contraste_adaptativo(imagem, c, n, modo)` calcula a média local $m$ convoluindo a imagem com a máscara da média $n \times n$, e a variância local por $\sigma^2 = \overline{f^2} - m^2$, convoluindo também a imagem ao quadrado. Pequenos valores negativos de $\sigma^2$, causados por arredondamento numérico, são zerados. Nos pixels com $\sigma > 0$ aplica-se a equação do contraste adaptativo; nos pixels com $\sigma = 0$ (região homogênea) o valor original é mantido, evitando a divisão por zero.

# 4 Resultados

## 4.1 Utilização da biblioteca

O sistema é utilizado a partir do Octave, chamando as funções da biblioteca. Um exemplo de uso:

```
addpath('src');
f = imread('images/cameraman.tif');

g = potencia_gamma(f, 1, 0.5);
r = bordas_sobel(f, 'repetir');      % r.gx, r.gy, r.magnitude, r.final
a = agucamento_laplaciano(f, 8);     % a.laplaciano_visual, a.final
c = convolucao(f, [0 0 0; 0 1 0; 0 0 -1], 'repetir', 128);
imwrite(ajustar_intervalo(c), 'relevo.png');
```

O comando `make resultados` executa o script `gerar_resultados.m`, que produz todas as imagens apresentadas nesta seção na pasta `resultados/`. Exceto quando indicado, as operações de vizinhança usam o modo de borda `repetir`.

## 4.2 Transformações de intensidade e histogramas

A Figura 2 mostra o negativo, a limiarização, a transformação logarítmica e o alargamento de contraste. O negativo da mamografia torna o tecido escuro e a lesão clara em um fundo branco, o que facilita observar as estruturas finas. A limiarização com $t = 128$ separa as cristas da impressão digital do fundo. A transformação logarítmica revela, na imagem do Hubble, objetos de baixa intensidade que estavam praticamente invisíveis, ao custo de clarear todo o fundo. No alargamento, com $(r_1, s_1) = (140, 30)$ e $(r_2, s_2) = (200, 225)$, o trecho central tem inclinação $195/60 \approx 3{,}3$: a faixa $[140, 200]$, que concentra quase todos os pixels da imagem de polímeros, passa a ocupar $[30, 225]$, e as estruturas tornam-se nítidas.

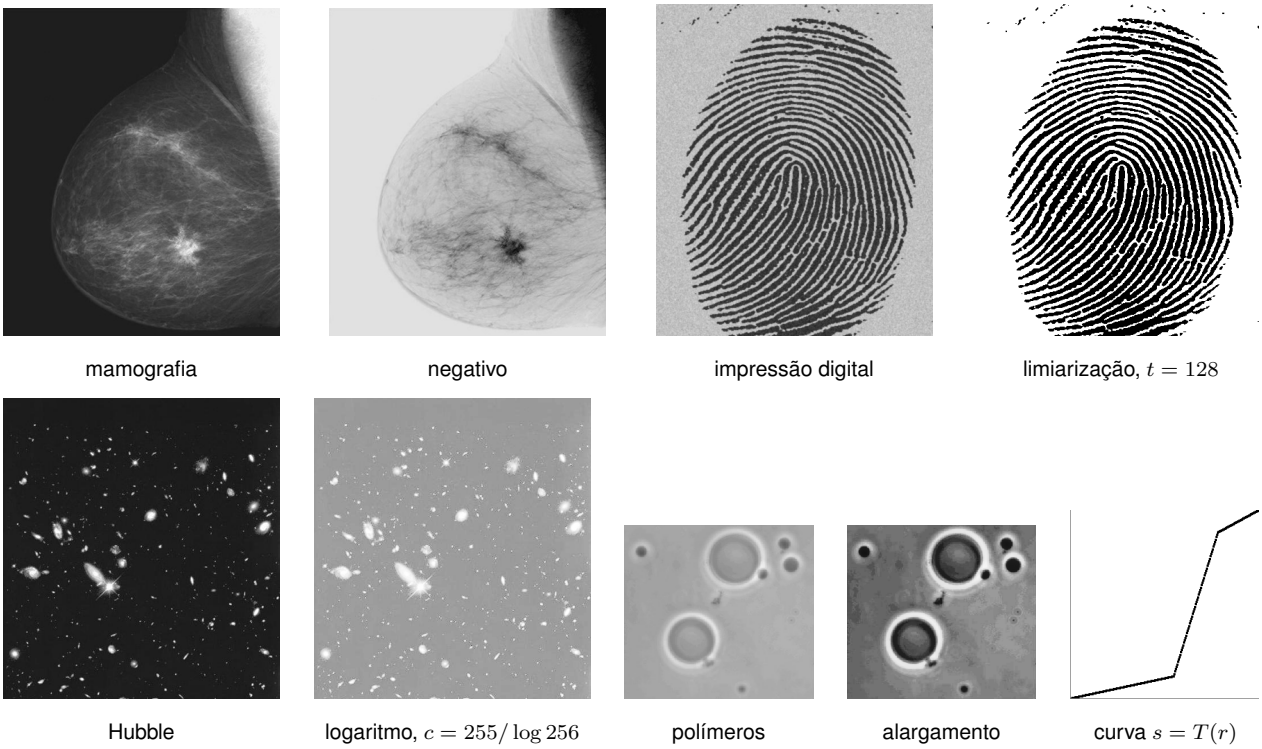



**Figura 2:** Negativo, limiarização, transformação logarítmica e alargamento de contraste.

A Figura 3 mostra a transformação de potência sobre uma imagem de ressonância predominantemente escura, com $c = 1$. Com $\gamma = 0{,}5$ e $\gamma = 0{,}3$ a coluna vertebral e os tecidos ao redor tornam-se visíveis, e quanto menor $\gamma$ maior o clareamento. Com $\gamma = 2$ a imagem escurece e apenas as regiões mais claras permanecem visíveis.

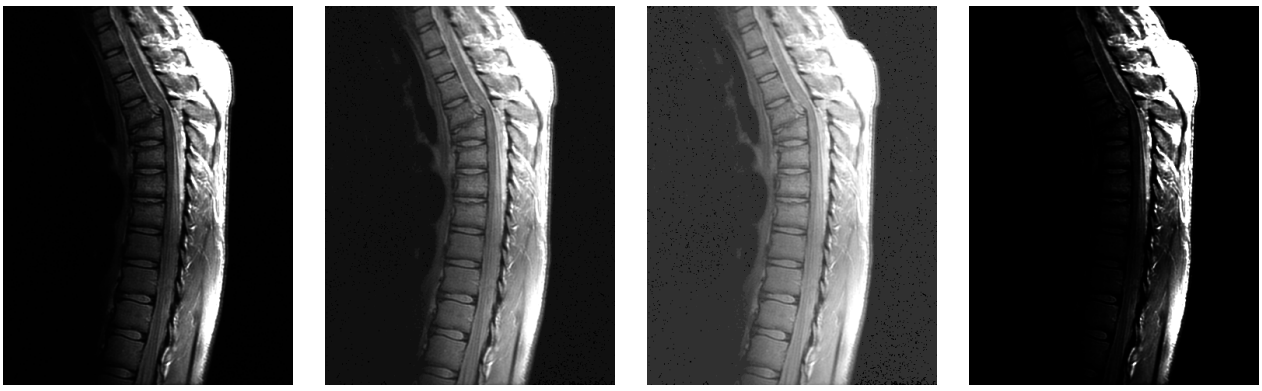



**Figura 3:** Transformação de potência com $c = 1$.

A Figura 4 mostra o dissolve cruzado. No uniforme, $t = 0{,}25$ mantém a imagem do cinegrafista dominante e $t = 0{,}75$ privilegia a Lena. No não uniforme, a matriz de pesos em rampa produz uma transição gradual da esquerda para a direita, e a matriz circular insere a Lena dentro de um círculo.

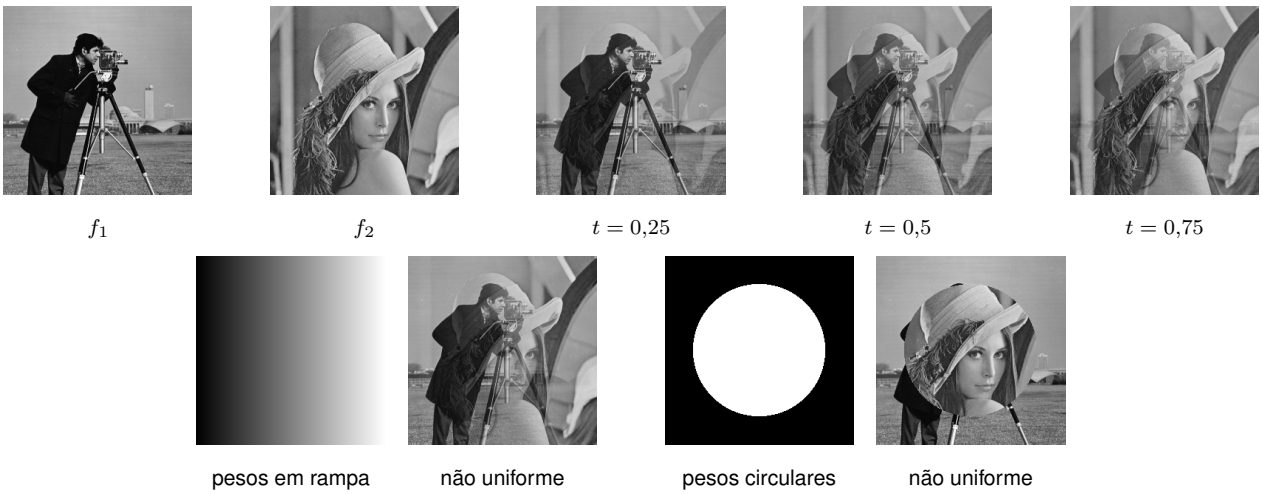



**Figura 4:** Dissolve cruzado uniforme (linha superior) e não uniforme (linha inferior).

A Figura 5 compara expansão e equalização. Na imagem escura, que ocupa apenas os níveis 2 a 106, a expansão estica o histograma para $[0, 255]$ e o desvio padrão passa de 12,9 para 31,5; a equalização vai além e redistribui os níveis, levando o desvio padrão a 70,7. Na imagem de polímeros, alguns poucos pixels já ocupam os extremos 90 e 241, por isso a expansão tem efeito modesto (desvio padrão de 13,8 para 23,3), enquanto a equalização produz um realce forte (73,0). Nos histogramas equalizados aparecem lacunas: pixels de mesmo nível não podem ser separados, então o número de níveis distintos diminui (de 152 para 69 nos polímeros).

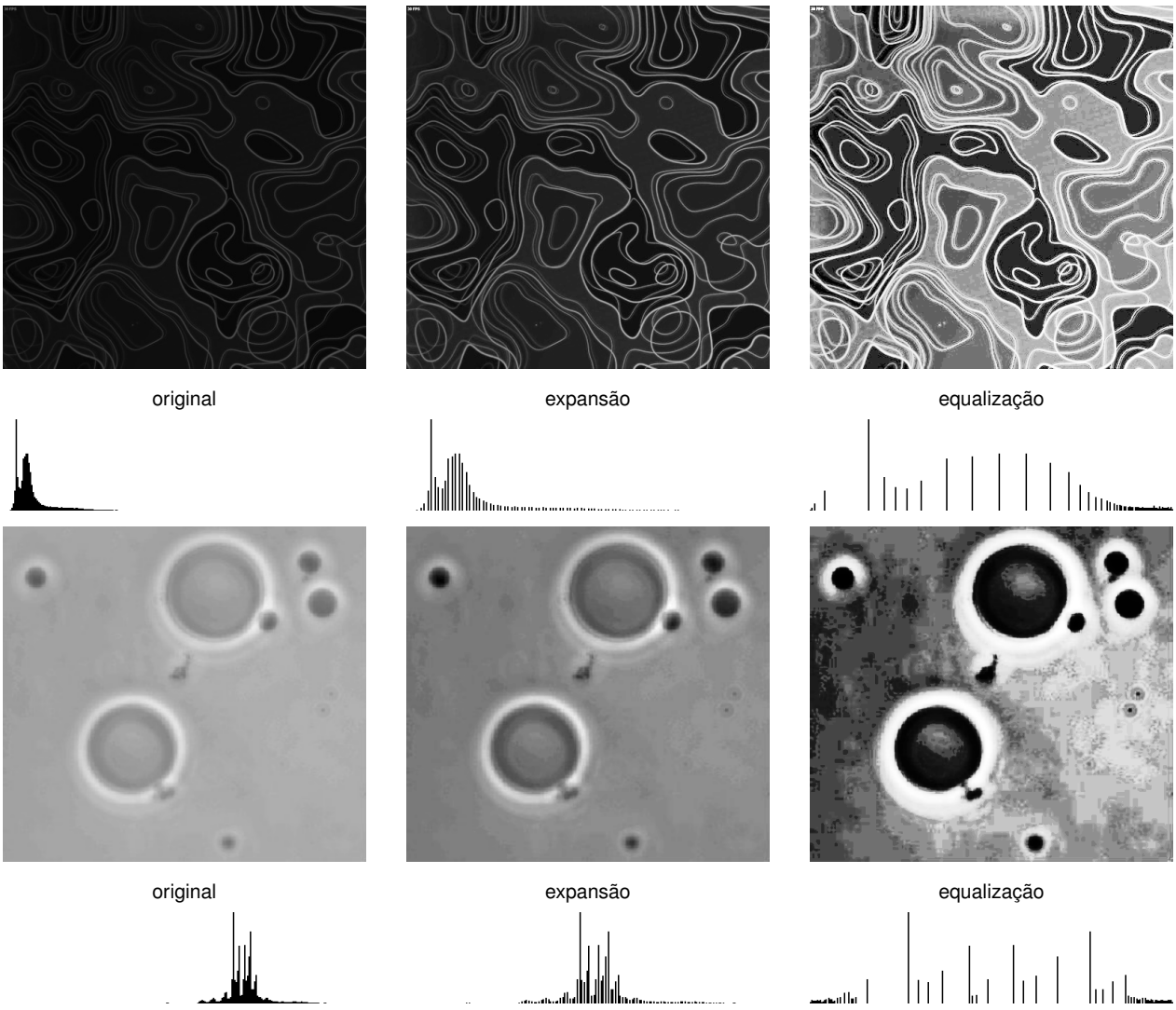


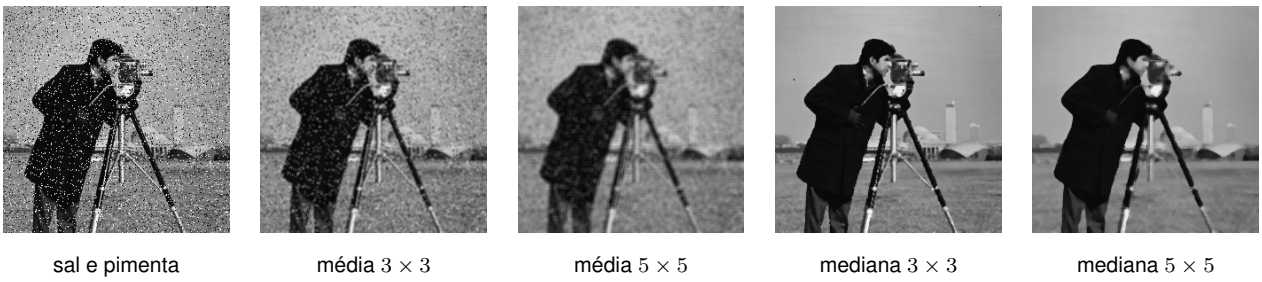

**Figura 5:** Expansão e equalização de histograma, com os respectivos histogramas abaixo de cada imagem.

## 4.3 Filtragem espacial

Para comparar os filtros, foram adicionados à imagem do cinegrafista um ruído impulsivo (sal e pimenta, 10% dos pixels) e um ruído gaussiano (desvio padrão 20). A Tabela 1 apresenta a raiz do erro quadrático médio (RMSE) em relação à imagem sem ruído, e a Figura 6 mostra os resultados.

| | Sal e pimenta (RMSE 44,5) | | Gaussiano (RMSE 19,2) | |
|---|---|---|---|---|
| Vizinhança | Média | Mediana | Média | Mediana |
| $3 \times 3$ | 20,8 | **12,3** | 14,5 | **14,0** |
| $5 \times 5$ | 21,3 | 16,8 | 18,5 | 17,1 |

**Tabela 1:** RMSE em relação à imagem original sem ruído (quanto menor, melhor).



sal e pimenta | média $3 \times 3$ | média $5 \times 5$ | mediana $3 \times 3$ | mediana $5 \times 5$



gaussiano | média $3 \times 3$ | média $5 \times 5$ | mediana $3 \times 3$ | mediana $5 \times 5$

**Figura 6:** Filtros da média e da mediana sobre ruído impulsivo (acima) e gaussiano (abaixo).

No ruído impulsivo a mediana $3 \times 3$ remove praticamente todos os pontos e preserva as bordas, enquanto a média apenas espalha os impulsos em manchas cinzentas e borra a imagem. No ruído gaussiano os dois filtros têm desempenho próximo. Em ambos os casos, aumentar a vizinhança para $5 \times 5$ suaviza mais, mas apaga detalhes finos (como o tripé e os prédios ao fundo), e o erro em relação à imagem original aumenta.

## 4.4 Testes de convolução

Todos os testes usam a função `convolucao` sobre a imagem do cinegrafista, com borda `repetir`. A Tabela 2 resume o intervalo dos valores obtidos antes do ajuste para $[0, 255]$ e a fração de pixels que saíram desse intervalo. Exceto quando indicado, os valores fora do intervalo foram saturados.

| Máscara | Soma | Mínimo | Máximo | $< 0$ | $> 255$ |
|---|---|---|---|---|---|
| 2.1, $c = 1$, $d = 1$ | 1 | $-1091$ | 1481 | 11,4% | 7,2% |
| 2.1, $c = 2$, $d = 1$ | 1 | $-2206$ | 2720 | 17,5% | 11,8% |
| 2.1, $c = 1$, $d = 3$ | 3 | $-1043$ | 1965 | 5,2% | 69,6% |
| 2.1, $c = 3$, $d = 3$ | 3 | $-3273$ | 4443 | 11,4% | 66,1% |
| 2.2, $h_1$, offset 128 | 0 | $-91$ | 349 | 0,6% | 0,5% |
| 2.2, $h_2$, offset 128 | 0 | $-280$ | 571 | 2,7% | 2,5% |
| 2.2, $h_1$, sem offset | 0 | $-219$ | 221 | 44,2% | 0,0% |
| 2.3, $h_3$ | 2 | $-300$ | 790 | 1,7% | 59,8% |
| 2.3, $h_4$ | $-8$ | $-2003$ | $-58$ | 100,0% | 0,0% |

**Tabela 2:** Valores da convolução antes do ajuste para $[0, 255]$.

### 4.4.1 Filtro de aguçamento

A máscara do teste 2.1 pode ser escrita como $g = d\,f - c\,\nabla^2_8 f$, em que $\nabla^2_8$ é o Laplaciano de vizinhança-8: o coeficiente central $8c + d$ é a soma de $d$ vezes o pixel com $c$ vezes a diferença entre o pixel e seus oito vizinhos. Assim, $c$ controla a intensidade do realce de bordas e $d$ controla o ganho de brilho, pois a soma dos coeficientes é exatamente $d$.

Com $c = d = 1$ a soma é 1: o brilho médio é preservado e as bordas ficam realçadas, junto com a textura da grama e o ruído (Figura 7). Com $c = 2$ e $d = 1$ o realce dobra e mais pixels saturam.

Com $d = 3$ a soma é 3 e o brilho triplica: o céu e a grama saturam em branco (cerca de 70% dos pixels acima de 255) e restam visíveis apenas os objetos escuros.

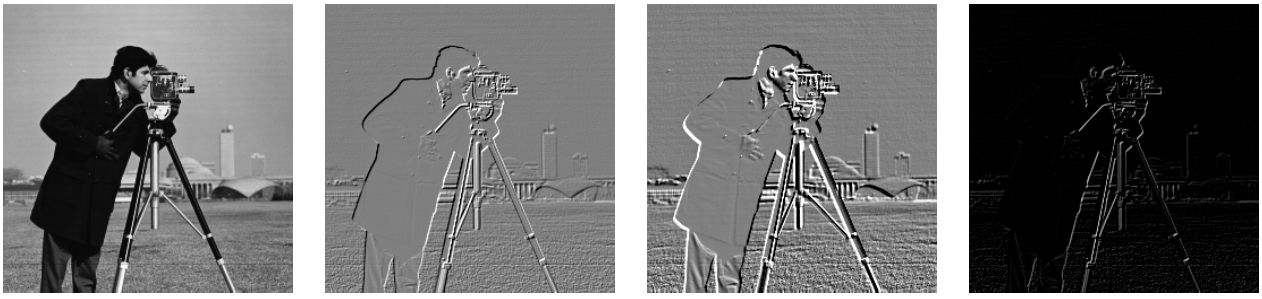


original | $c = 1, d = 1$ | $c = 2, d = 1$ | $c = 1, d = 3$ | $c = 3, d = 3$

**Figura 7:** Teste 2.1: filtro de aguçamento com diferentes valores de $c$ e $d$.

### 4.4.2 Filtros de relevo

As duas máscaras somam zero: regiões homogêneas resultam em 0, que o offset 128 transforma em cinza médio, e apenas as transições aparecem, claras de um lado e escuras do outro, como se a imagem fosse iluminada lateralmente (Figura 8). Após a rotação de $180°$ da convolução, $h_1$ calcula $f(x, y) - f(x - 1, y - 1)$, uma diferença ao longo da diagonal principal, enquanto $h_2$ combina pixels ao longo da diagonal secundária com sinais invertidos. Por isso a "iluminação" vem de lados diferentes: o contorno esquerdo do casaco aparece escuro com $h_1$ e claro com $h_2$. O coeficiente 2 de $h_2$ produz um relevo mais intenso. Sem o offset, 44% dos pixels ficam negativos e são saturados em 0, e só um dos lados de cada borda permanece visível, o que mostra a função do offset.



original | $h_1$, offset 128 | $h_2$, offset 128 | $h_1$, sem offset

**Figura 8:** Teste 2.2: filtros de relevo.

### 4.4.3 Máscaras de detecção de bordas

A máscara $h_3$ tem coeficientes negativos acima e à esquerda e positivos abaixo e à direita, o que a torna sensível a transições na direção diagonal. Porém, sua soma é 2, e não 0: regiões homogêneas são multiplicadas por 2, o céu satura em branco (59,8% dos pixels acima de 255) e o resultado é a imagem clareada com as bordas diagonais reforçadas, e não um mapa de bordas.

A máscara $h_4$ lembra o Laplaciano, mas todos os seus coeficientes são negativos: ela calcula $-(4f(x, y) +$ soma dos 4 vizinhos$)$, isto é, uma média ponderada multiplicada por $-8$. Trata-se de um filtro passa-baixas invertido, e não de um detector de bordas. Todos os valores resultam negativos: saturados, produzem uma imagem totalmente preta; em módulo, quase todos ultrapassam 255; normalizados, revelam o negativo suavizado da imagem (Figura 9). O teste mostra que os sinais dos coeficientes determinam o comportamento do filtro: basta inverter o sinal dos vizinhos para transformar um detector de bordas em um filtro de suavização.

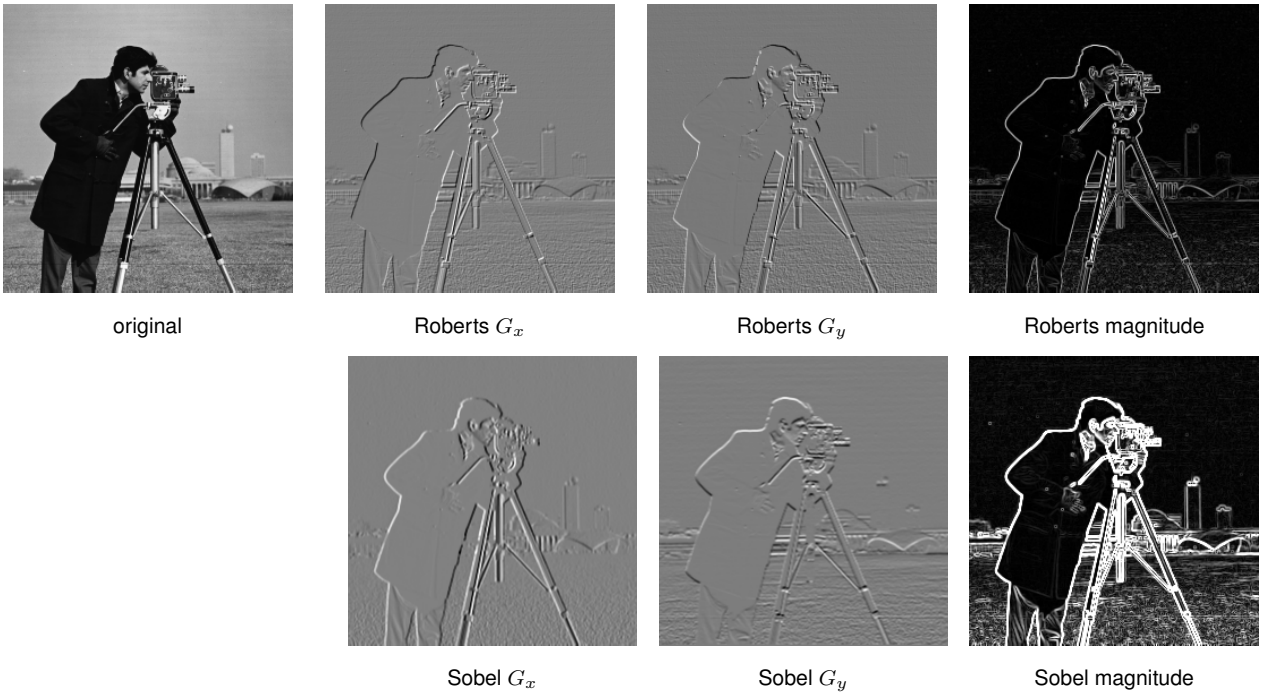


$h_3$, saturado | $h_3$, normalizado | $h_4$, módulo | $h_4$, normalizado

**Figura 9:** Teste 2.3: máscaras $h_3$ e $h_4$ com diferentes tratamentos do intervalo. A versão saturada de $h_4$ é inteiramente preta e foi omitida.

## 4.5 Roberts, Sobel, Kirsch e Laplaciano

A Figura 10 mostra as respostas intermediárias dos operadores de gradiente. $G_x$ e $G_y$ foram normalizados para exibição (cinza médio corresponde a zero). Em $G_x$ destacam-se as bordas verticais, como as pernas do tripé; em $G_y$, as horizontais, como a linha do horizonte. O Roberts produz bordas finas e ruidosas; o Sobel produz bordas mais espessas e contínuas, graças à suavização embutida.



original | Roberts $G_x$ | Roberts $G_y$ | Roberts magnitude

Sobel $G_x$ | Sobel $G_y$ | Sobel magnitude

**Figura 10:** Operadores de Roberts e Sobel: respostas de cada máscara e magnitude do gradiente.

A Figura 11 mostra as oito respostas do Kirsch, com valores negativos saturados em zero. Cada máscara realça as bordas de uma orientação, e o máximo entre elas reúne as bordas de todas as direções. A resposta final é a mais intensa entre os operadores, pois os coeficientes 5 e $-3$ produzem valores elevados, e muitas bordas saturam.

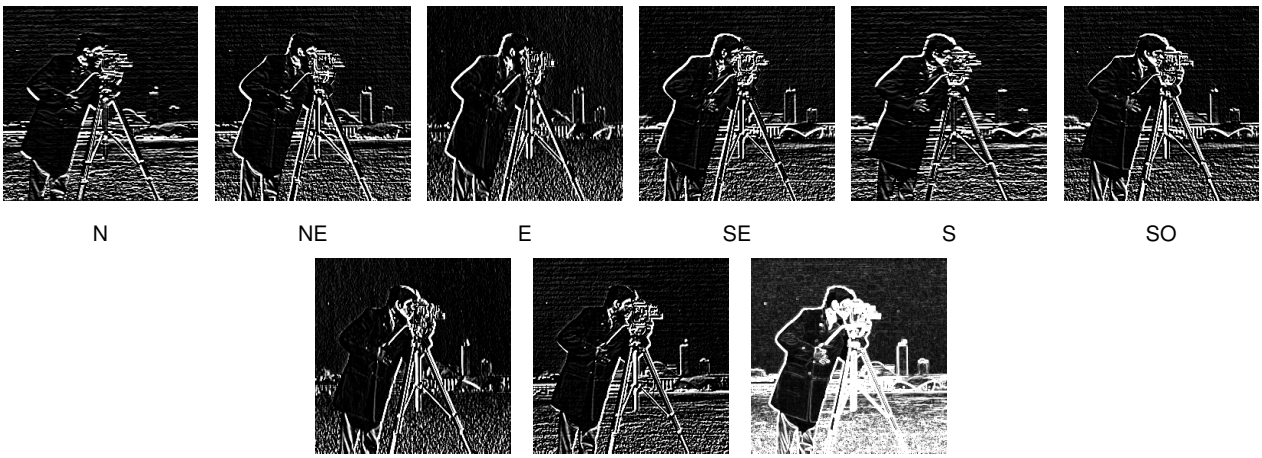



**Figura 11:** Operador de Kirsch: resposta de cada uma das oito máscaras e resposta final.

A Figura 12 mostra o Laplaciano. Na resposta normalizada, regiões planas ficam em cinza médio e cada borda aparece como um par de linhas, uma clara e outra escura. Em módulo, as bordas surgem duplicadas e a textura da grama gera muitas respostas, o que evidencia a sensibilidade ao ruído. A máscara de vizinhança-8 produz respostas mais fortes que a de vizinhança-4.

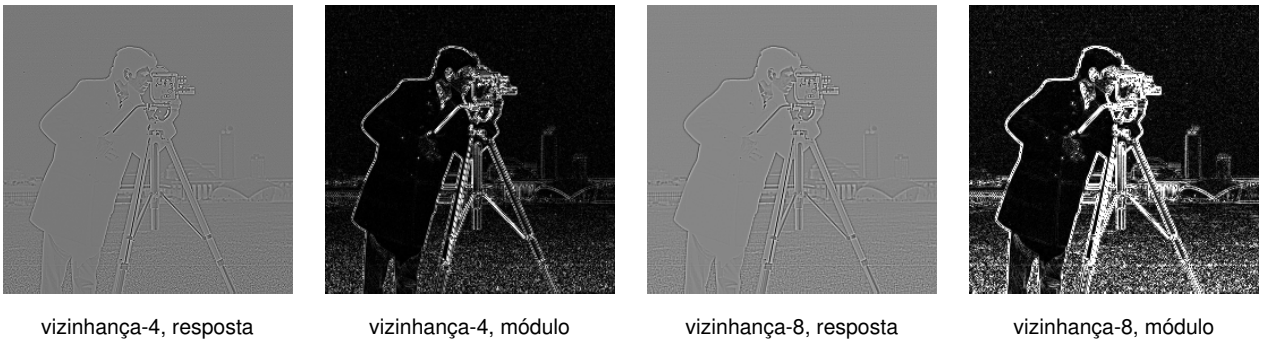
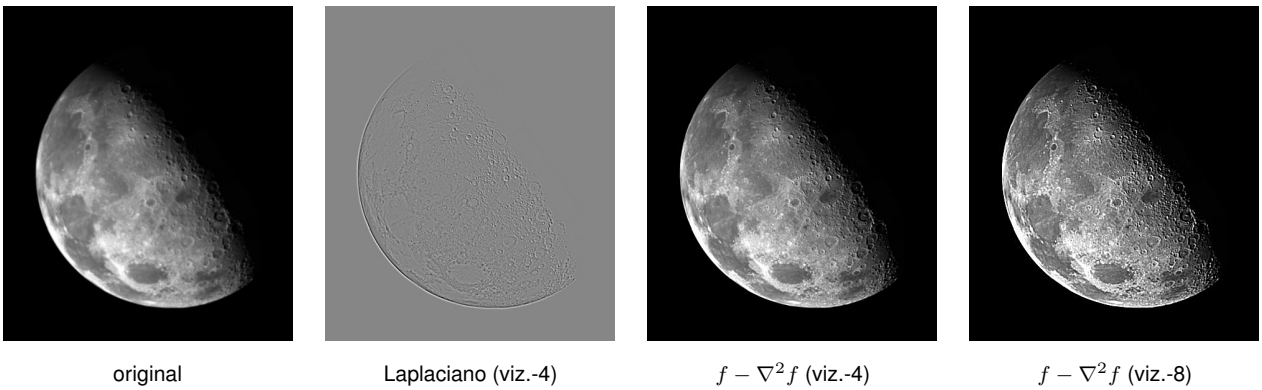



**Figura 12:** Operador Laplaciano: resposta com sinal (normalizada) e em módulo.

## 4.6 Aguçamento pelo Laplaciano e high boost

A Figura 13 apresenta separadamente a resposta do Laplaciano e a imagem combinada $f - \nabla^2 f$. As crateras e o relevo da Lua ficam mais definidos, e a vizinhança-8 produz um realce mais forte. No high boost, com média $5 \times 5$, a máscara de detalhe contém apenas as variações locais; com $k = 1$ o realce é moderado, e com $k = 4$ as bordas ficam acentuadas e o ruído da imagem começa a se destacar.



original | Laplaciano (viz.-4) | $f - \nabla^2 f$ (viz.-4) | $f - \nabla^2 f$ (viz.-8)

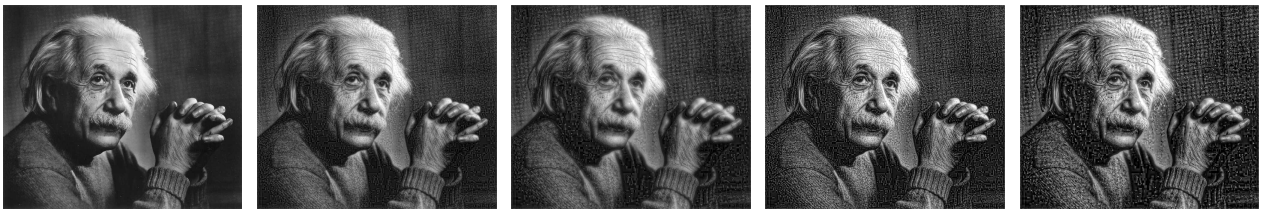


detalhe $f - \bar{f}$ | high boost, $k = 1$ | high boost, $k = 2$ | high boost, $k = 4$

**Figura 13:** Aguçamento pelo Laplaciano (acima) e high boost com média $5 \times 5$ (abaixo).

### 4.7 Controle de contraste adaptativo

A Figura 14 mostra o contraste adaptativo com $c \in \{20, 40\}$ e $n \in \{7, 15\}$. As rugas, os cabelos e a textura do fundo, pouco visíveis na imagem original, tornam-se evidentes. Aumentar $c$ aumenta o contraste local em toda a imagem. Aumentar $n$ torna as estatísticas menos locais: com $n = 7$ o realce acompanha detalhes muito finos e amplifica o ruído; com $n = 15$ o resultado é mais suave e preserva melhor as regiões grandes.



original | $c = 20$, $n = 7$ | $c = 20$, $n = 15$ | $c = 40$, $n = 7$ | $c = 40$, $n = 15$

**Figura 14:** Controle de contraste adaptativo.

## 5 Discussão

**Média e mediana.** Os resultados confirmam a diferença teórica entre um filtro linear e um não linear. A média distribui a energia de um impulso por toda a vizinhança, de modo que um ponto de valor 255 cercado de 100 resulta em 117 com uma máscara $3 \times 3$; a mediana descarta o valor extremo e devolve 100. Contra o ruído impulsivo, a mediana teve RMSE 41% menor que a média ($12{,}3$ contra $20{,}8$). Contra o ruído gaussiano, em que todos os pixels são levemente perturbados, as duas abordagens se equivalem.

**Operadores de borda.** O Roberts, com máscaras $2 \times 2$, localiza bem as bordas, mas é o mais

sensível ao ruído, pois cada resposta depende de apenas dois pixels. O Sobel suaviza na direção perpendicular à derivada e produz bordas mais contínuas. O Kirsch avalia oito direções e responde de forma intensa em todas elas, com o maior custo computacional (oito convoluções). O Laplaciano usa uma única máscara isotrópica, mas, por ser de segunda ordem, é o mais afetado pela textura da grama e gera bordas duplas.

**Bordas e aguçamento.** Os detectores devolvem apenas as transições e descartam o conteúdo das regiões homogêneas, enquanto o aguçamento adiciona as transições à imagem original e preserva a cena. O teste 2.1 mostra que essas operações estão ligadas: a mesma máscara pode ser lida como "$d$ vezes a imagem menos $c$ vezes o Laplaciano".

**Expansão e equalização.** A expansão depende apenas do mínimo e do máximo, e poucos pixels extremos bastam para anulá-la, como ocorreu com a imagem de polímeros. A equalização usa a distribuição completa dos níveis e produz realce forte em ambos os casos, mas também amplifica variações em regiões homogêneas e reduz o número de níveis distintos.

**Influência dos parâmetros e das bordas.** Nos testes de convolução, a soma dos coeficientes antecipou corretamente o comportamento de cada máscara: preservação de brilho (soma 1), ganho de brilho (soma 3), relevo (soma 0) e inversão (soma $-8$). O tratamento das bordas influencia apenas uma faixa estreita junto às margens; com preenchimento por zeros, os detectores de borda acusam uma moldura artificial ao redor da imagem, motivo pelo qual os resultados usaram o modo `repetir`.

## 5.1 Questões de domínio conceitual

**1. Transformação pontual e operação de vizinhança.** Em uma transformação pontual, o valor de saída de um pixel depende apenas do valor desse mesmo pixel: $s = T(r)$. Em uma operação de vizinhança, depende também dos pixels ao redor. São pontuais: o negativo ($255 - r$, que usa só o próprio pixel), a limiarização (compara o próprio pixel com $t$), a transformação gamma e a equalização de histograma (a tabela é calculada a partir da imagem toda, mas cada pixel é transformado apenas pelo seu próprio valor). São de vizinhança: os filtros da média e da mediana, os detectores de bordas e o contraste adaptativo, que usam a janela $n \times n$ ao redor do pixel.

**2. Comportamento de $s = c\,r^{\gamma}$.** Com $r$ normalizado em $[0, 1]$: para $\gamma < 1$ a curva fica acima da identidade, os tons escuros são expandidos e a imagem clareia; para $\gamma = 1$ (e $c = 1$) a transformação é a identidade; para $\gamma > 1$ a curva fica abaixo da identidade, os tons claros são expandidos e a imagem escurece. Em uma imagem predominantemente escura, passar $\gamma$ de 1 para 0,5 deve clarear a imagem e revelar detalhes nas sombras, com algum aumento de ruído e redução do contraste nas regiões claras. Foi o que se observou na Figura 3: um pixel de valor 64 passa a 128.

**3. Média e mediana.** A média é linear (é uma convolução), calculada como a soma ponderada dos valores da janela; a mediana é não linear, calculada pela ordenação dos valores e escolha do elemento central. Diante de ruído impulsivo, a média espalha o impulso pela vizinhança e borra a imagem, enquanto a mediana descarta o valor extremo e preserva as bordas, como mostram a Figura 6 e a Tabela 1.

**4. $G_x$ e $G_y$ no Sobel.** $G_x$ estima a derivada horizontal, respondendo a bordas verticais; $G_y$ estima a derivada vertical, respondendo a bordas horizontais. Juntos formam o vetor gradiente, e a magnitude $\sqrt{G_x^2 + G_y^2}$ fornece a intensidade da borda independentemente da sua orientação. No centro da matriz $\left[\begin{smallmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{smallmatrix}\right]$, o sistema obtém $G_x = -80$, $G_y = -240$ e magnitude $\sqrt{64000} \approx 253$.

**5. Comparação dos operadores.**

| Operador | Tamanho | Máscaras | Direções | Sensibilidade ao ruído |
|---|---|---|---|---|
| Roberts | $2 \times 2$ | 2 | diagonais | alta: usa apenas 2 pixels por resposta |
| Sobel | $3 \times 3$ | 2 | horizontal e vertical | moderada: suaviza na direção perpendicular |
| Kirsch | $3 \times 3$ | 8 | 8 direções, a cada $45°$ | moderada: cada máscara soma 3 pixels de cada lado |
| Laplaciano | $3 \times 3$ | 1 | isotrópico | alta: derivada de segunda ordem |

**6. Múltiplas máscaras no Kirsch.** Cada máscara responde fortemente apenas às bordas de uma orientação. Usar oito orientações garante que qualquer borda tenha ao menos uma máscara aproximadamente alinhada a ela. Tomar a maior resposta entre as máscaras seleciona a orientação que melhor descreve a borda naquele pixel, e esse valor representa a intensidade da borda; o índice da máscara vencedora indica sua direção.

**7. Laplaciano em bordas e aguçamento.** O Laplaciano é nulo em regiões homogêneas e em variações lineares, e responde com sinais opostos de cada lado de uma transição; por isso serve para evidenciar bordas. No aguçamento, porém, a resposta isolada perderia o conteúdo da imagem, então ela é combinada com a original para reforçar as transições sem descartar a cena. Quando o coeficiente central da máscara é negativo, a resposta deve ser subtraída ($g = f - \nabla^2 f$); quando é positivo, deve ser somada. Em ambos os casos, o objetivo é aumentar a diferença entre o pixel e seus vizinhos.

**8. Valores fora de $[0, 255]$.** Coeficientes negativos produzem somas negativas, e coeficientes cuja soma é maior que 1 produzem valores acima de 255; na Tabela 2, os valores chegaram a $-3273$ e $4443$. O sistema calcula a convolução em `double` e só então converte o resultado com `ajustar_intervalo`, saturando, normalizando ou aplicando o módulo. Converter diretamente para inteiro de 8 bits pode ter dois efeitos, dependendo da linguagem: em C ou em Python/NumPy ocorre *wrap-around* (300 vira 44, e $-50$ vira 206), o que gera artefatos sem sentido; no Octave, `uint8()` satura, e a informação além dos limites é perdida (com $h_4$ a imagem fica toda preta). Além disso, fazer as contas intermediárias em `uint8` saturaria cada soma parcial e produziria resultados incorretos mesmo quando o valor final coubesse no intervalo.

**9. Expansão e equalização.** A expansão é uma transformação linear que leva $[r_{\min}, r_{\max}]$ a $[0, 255]$ e preserva a forma do histograma. A equalização é não linear e baseada na distribuição acumulada: afasta os níveis mais frequentes e tende a uniformizar o histograma. A equalização não produz necessariamente uma imagem melhor: ela amplifica o ruído em regiões homogêneas, pode dar aparência artificial, reduz o número de níveis distintos e, em uma imagem constante, leva todos os pixels a 255.

**10. Aguçamento e detecção de bordas.** A detecção de bordas produz uma imagem que contém apenas as transições, com as regiões homogêneas em preto. O aguçamento produz a imagem original com as transições reforçadas. Uma imagem gerada por um operador de bordas não é, portanto, uma imagem aguçada: ela é o ingrediente que, somado ou subtraído da original, produz o aguçamento.

## 5.2 Limitações e dificuldades encontradas

- A convolução percorre a imagem pixel a pixel, o que é lento em Octave: gerar todos os resultados deste relatório leva cerca de um minuto. Uma implementação vetorizada seria mais rápida, mas menos legível.
- Máscaras de dimensão par, como as de Roberts, não têm um centro único. Foi adotado o centro na posição $\lceil n/2 \rceil$, o que exigiu preenchimento assimétrico das bordas.

- A convolução gira a máscara em $180°$, o que inverte o sinal de $G_x$ e $G_y$ e troca os rótulos de direção do Kirsch em relação à intuição de quem lê as máscaras como estão escritas.
- O nome `gamma` entrava em conflito com a função $\Gamma$ do Octave, o que quebrava funções como `factorial`; a função foi renomeada para `potencia_gamma`.
- Na primeira versão, contas feitas em `uint8` saturavam e arredondavam valores intermediários; por isso todas as operações passaram a converter a imagem para `double` antes de calcular.
- A formulação do contraste adaptativo adotada amplifica o ruído em regiões quase homogêneas, onde $\sigma$ é muito pequeno.

# 6 Conclusão

Foi desenvolvida uma biblioteca em GNU Octave com 22 funções que implementam todas as operações pedidas na avaliação: operações algébricas, transformações de intensidade, histograma, contraste adaptativo, filtros espaciais, convolução genérica, detectores de bordas e aguçamento. Nenhum algoritmo avaliado utiliza funções prontas de processamento, e todas as funções são verificadas por 184 testes unitários com valores calculados manualmente.

A convolução genérica, com tratamento explícito das bordas e dos valores fora de $[0, 255]$, mostrou-se a base do sistema: filtros, detectores de bordas e técnicas de aguçamento são todos aplicações dela com máscaras diferentes. Os testes obrigatórios evidenciaram que a soma e a disposição dos coeficientes determinam o efeito de uma máscara, a ponto de uma máscara aparentemente de bordas ($h_4$) se revelar um filtro de suavização invertido.

Entre as limitações permanecem o desempenho da convolução pixel a pixel e a dependência do contraste adaptativo em relação ao ruído. Como trabalhos futuros, seria possível vetorizar a convolução e acrescentar outros modos de tratamento de bordas, como o espelhamento.

# 7 Declaração de uso de ferramentas de Inteligência Artificial

**Ferramentas utilizadas:** Claude Code (Anthropic), assistente de programação baseado em IA generativa.

**Finalidade do uso:** configuração do ambiente de desenvolvimento (Nix, MISS_HIT, Makefile e executor de testes); explicação dos conceitos e das opções de implementação de cada operação; revisão e correção de erros nas funções escritas pelo aluno; escrita dos testes unitários; implementação de parte das funções da biblioteca; geração do script de resultados; e redação da primeira versão deste relatório.

**Observações:** As funções `preencher_bordas`, `convolucao`, `negativo`, `limiarizacao`, `potencia_gamma` e `logaritimo` foram iniciadas pelo aluno e corrigidas ou ampliadas com auxílio da ferramenta. As demais funções foram implementadas com auxílio da ferramenta e revisadas pelo aluno. Todas as decisões de projeto foram discutidas durante o desenvolvimento, e o aluno é responsável por todo o código, texto e resultados apresentados.

# 8 Referências


- GONZALEZ, Rafael C.; WOODS, Richard E. **Processamento digital de imagens**. 3. ed. São Paulo: Pearson Prentice Hall, 2010.

- MARQUES FILHO, Ogê; VIEIRA NETO, Hugo. **Processamento digital de imagens**. Rio de Janeiro: Brasport, 1999.
- KIRSCH, Russell A. Computer determination of the constituent structure of biological images. **Computers and Biomedical Research**, v. 4, n. 3, p. 315–328, 1971.
- ROBERTS, Lawrence G. **Machine perception of three-dimensional solids**. Tese (Doutorado) – Massachusetts Institute of Technology, Cambridge, 1963.
- EATON, John W. et al. **GNU Octave manual**. Disponível em: https://docs.octave.org. Acesso em: out. 2026.
- MISS_HIT. **MISS_HIT: MATLAB independent, small & safe, high integrity tools**. Disponível em: https://misshit.org. Acesso em: out. 2026.